Vertex Generative AI SDK for Python

Version latest

The Gen AI Modules in the Vertex SDK help developers use Google's generative AI Gemini models (http://cloud.google.com/vertex-ai/docs/generative-ai/multimodal/overview) to build AI-powered features and applications in Vertex.

The modules currently available are: Evaluation, Agent Engines, Prompt Management, and Prompt Optimization. See below for instructions on getting started with each module. For other Gemini features on Vertex, use the Gen AI SDK (https://github.com/googleapis/python-genai).

Installation

To install the google-cloud-aiplatform (https://pypi.org/project/google-cloud-aiplatform/) Python package, run the following command:

```
pip3 install --upgrade --user "google-cloud-aiplatform>=1.114.0"
```

Imports:

`import vertexai` (https://docs.cloud.google.com/python/docs/reference/vertexai/latest)
`from vertexai import types` (https://docs.cloud.google.com/python/docs/reference/vertexai/late

Client initialization

```
client = vertexai.Client(project='my-project', location='us-central1')
```

Gen AI Evaluation

To run evaluation, first generate model responses from a set of prompts.

```
import pandas as pd

prompts_df = pd.DataFrame({
    "prompt": [
        "What is the capital of France?",
        "Write a haiku about a cat.",
        "Write a Python function to calculate the factorial of a number.",
        "Translate 'How are you?' to French.",
    ],
})

inference_results = client.evals.run_inference(
    model="gemini-2.5-flash",
        ("def factorial(n):\n"
         "    if n < 0:\n"
         "        return 'Factorial does not exist for negative numbers'\n"
         "    elif n == 0:\n"
         "        return 1\n"
         "    else:\n"
         "        fact = 1\n"
         "        i = 1\n"
         "        while i <= n:\n"
         "            fact *= i\n"
         "            i += 1\n"
         "        return fact"),
)
inference_results.show()
```

Then run evaluation by providing the inference results and specifying the metric types.

```
eval_result = client.evals.evaluate(
    dataset=inference_results,
    metrics=[
        types.RubricMetric.GENERAL_QUALITY,
    ]
)
eval_result.show()
```

Agent Engine with Agent Development Kit (ADK)

First, define a function that looks up the exchange rate:

```
def get_exchange_rate(
    currency_from: str = "USD",
    currency_to: str = "EUR",
    currency_date: str = "latest",
):
    """Retrieves the exchange rate between two currencies on a specified date

    Uses the Frankfurter API (https://api.frankfurter.app/) to obtain
    exchange rate data.

    Returns:
        dict: A dictionary containing the exchange rate information.
            Example: {"amount": 1.0, "base": "USD", "date": "2023-11-24",
                "rates": {"EUR": 0.95534}}
    """
    import requests
    response = requests.get(
        f"https://api.frankfurter.app/{currency_date}",
        params={"from": currency_from, "to": currency_to},
    )
    return response.json()
```

Next, define an ADK Agent:

```
from google.adk.agents import Agent
from vertexai.agent_engines import AdkApp

app = AdkApp(agent=Agent(
    model="gemini-2.0-flash",        # Required.
    name='currency_exchange_agent',  # Required.
    tools=[get_exchange_rate],       # Optional.
))
```

Test the agent locally using US dollars and Swedish Krona:

```
async for event in app.async_stream_query(
    user_id="user-id",
```

```
    message="What is the exchange rate from US dollars to SEK today?",
):
    print(event)
```

To deploy the agent to Agent Engine:

```
remote_app = client.agent_engines.create(
    agent=app,
    config={
        "requirements": ["google-cloud-aiplatform[agent_engines,adk]"],
    },
)
```

You can also run queries against the deployed agent:

```
async for event in remote_app.async_stream_query(
    user_id="user-id",
    message="What is the exchange rate from US dollars to SEK today?",
):
    print(event)
```

Prompt Optimization

To do a zero-shot prompt optimization, use the **`optimize_prompt`** method.

```
prompt = "Generate system instructions for a question-answering assistant"
response = client.prompt_optimizer.optimize_prompt(prompt=prompt)
print(response.raw_text_response)
if response.parsed_response:
    print(response.parsed_response.suggested_prompt)
```

To call the data-driven prompt optimization, call the **`optimize`** method. In this case however, we need to provide **`vapo_config`**. This config needs to have either service account or project **number** and the config path. Please refer to this tutorial (https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/data-driven-optimizer) for more details on config parameter.

```
import logging

project_number = PROJECT_NUMBER # replace with your project number
service_account = f"{project_number}-compute@developer.gserviceaccount.com"

vapo_config = types.PromptOptimizerVAPOConfig(
    config_path="gs://your-bucket/config.json",
    service_account_project_number=project_number,
    wait_for_completion=False
)

# Set up logging to see the progress of the optimization job
logging.basicConfig(encoding='utf-8', level=logging.INFO, force=True)

result = client.prompt_optimizer.optimize(method="vapo", config=vapo_config)
```

We can also call optimize method async.

```
await client.aio.prompt_optimizer.optimize(method="vapo", config=vapo_config)
```

Prompt Management

First define your prompt as a dictionary or `types.Prompt` object. Then call `create()`.

```
prompt = {
    "prompt_data": {
        "contents": [{"parts": [{"text": "Hello, {name}! How are you?"}]}],
        "system_instruction": {"parts": [{"text": "Please answer in a short se
        "variables": [
            {"name": {"text": "Alice"}},
        ],
        "model": "gemini-2.5-flash",
    },
}

prompt_resource = client.prompts.create(
    prompt=prompt,
)
```

Note that you can also use the `types.Prompt` object to define your prompt. Some of the types used to do this are from the Gen AI SDK (https://github.com/googleapis/python-genai).

```
import types
from google.genai import types as genai_types

prompt = types.Prompt(
    prompt_data=types.PromptData(
        contents=[genai_types.Content(parts=[genai_types.Part(text="Hello, {na
        system_instruction=genai_types.Content(parts=[genai_types.Part(text="I
        variables=[
            {"name": genai_types.Part(text="Alice")},
        ],
        model="gemini-2.5-flash",
    ),
)
```

Retrieve a prompt by calling `get()` with the `prompt_id`.

```
retrieved_prompt = client.prompts.get(prompt_id=prompt_resource.prompt_id)
```

After creating or retrieving a prompt, you can call `generate_content()` with that prompt using the Gen AI SDK.

The following uses a utility function available on Prompt objects to transform a Prompt object into a list of Content objects for use with `generate_content`. To run this you need to have the Gen AI SDK installed, which you can do via `pip install google-genai`.

```
from google import genai
from google.genai import types as genai_types

# Create a Client in the Gen AI SDK
genai_client = genai.Client(vertexai=True, project="your-project", location="

# Call generate_content() with the prompt
response = genai_client.models.generate_content(
    model=retrieved_prompt.prompt_data.model,
    contents=retrieved_prompt.assemble_contents(),
```

)

Warning

The following Generative AI modules in the Vertex AI SDK are deprecated as of June 24, 2025 and will be removed on June 24, 2026: **`vertexai.generative_models`**, **`vertexai.language_models`**, **`vertexai.vision_models`**, **`vertexai.tuning`**, **`vertexai.caching`**. Please use the Google Gen AI SDK (https://pypi.org/project/google-genai/) to access these features. See the migration guide (https://cloud.google.com/vertex-ai/generative-ai/docs/deprecations/genai-vertexai-sdk) for details. You can continue using all other Vertex AI SDK modules, as they are the recommended way to use the API.

Imports:

`import vertexai` (https://docs.cloud.google.com/python/docs/reference/vertexai/latest)

Initialization:

```
vertexai.init(project='my-project', location='us-central1')
```

Basic generation:

```
from vertexai.generative_models import GenerativeModel
model = GenerativeModel("gemini-pro")
print(model.generate_content("Why is sky blue?"))
```

Using images and videos

```
from vertexai.generative_models import GenerativeModel, Image
vision_model = GenerativeModel("gemini-pro-vision")

# Local image
image = Image.load_from_file("image.jpg")
print(vision_model.generate_content(["What is shown in this image?", image]))

# Image from Cloud Storage
image_part = generative_models.Part.from_uri("gs://download.tensorflow.org/ex
print(vision_model.generate_content([image_part, "Describe this image?"]))

# Text and video
video_part = Part.from_uri("gs://cloud-samples-data/video/animals.mp4", mime_
print(vision_model.generate_content(["What is in the video? ", video_part]))
```

Chat

```
from vertexai.generative_models import GenerativeModel, Image
vision_model = GenerativeModel("gemini-ultra-vision")
vision_chat = vision_model.start_chat()
image = Image.load_from_file("image.jpg")
print(vision_chat.send_message(["I like this image.", image]))
print(vision_chat.send_message("What things do I like?."))
```

System instructions

```
from vertexai.generative_models import GenerativeModel
model = GenerativeModel(
    "gemini-1.0-pro",
    system_instruction=[
        "Talk like a pirate.",
        "Don't use rude words.",
    ],
)
print(model.generate_content("Why is sky blue?"))
```

Function calling

```
# First, create tools that the model is can use to answer your questions.
# Describe a function by specifying it's schema (JsonSchema format)
get_current_weather_func = generative_models.FunctionDeclaration(
    name="get_current_weather",
    description="Get the current weather in a given location",
    parameters={
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "The city and state, e.g. San Francisco, CA"
            },
            "unit": {
                "type": "string",
                "enum": [
                    "celsius",
                    "fahrenheit",
                ]
            }
        },
        "required": [
            "location"
        ]
    },
)
# Tool is a collection of related functions
weather_tool = generative_models.Tool(
    function_declarations=[get_current_weather_func],
)

# Use tools in chat:
model = GenerativeModel(
    "gemini-pro",
    # You can specify tools when creating a model to avoid having to send ther
    tools=[weather_tool],
)
chat = model.start_chat()
# Send a message to the model. The model will respond with a function call.
print(chat.send_message("What is the weather like in Boston?"))
# Then send a function response to the model. The model will use it to answer
print(chat.send_message(
    Part.from_function_response(
        name="get_current_weather",
        response={
```

```
            "content": {"weather": "super nice"},
        }
    ),
))
```

Automatic Function calling

Note: The `FunctionDeclaration.from_func` converter does not support nested types for parameters. Please provide full `FunctionDeclaration` instead.

```
from vertexai.preview.generative_models import GenerativeModel, Tool, Functio

# First, create functions that the model can use to answer your questions.
def get_current_weather(location: str, unit: str = "centigrade"):
    """Gets weather in the specified location.

    Args:
        location: The location for which to get the weather.
        unit: Optional. Temperature unit. Can be Centigrade or Fahrenheit. De
    """
    return dict(
        location=location,
        unit=unit,
        weather="Super nice, but maybe a bit hot.",
    )

# Infer function schema
get_current_weather_func = FunctionDeclaration.from_func(get_current_weather)
# Tool is a collection of related functions
weather_tool = Tool(
    function_declarations=[get_current_weather_func],
)

# Use tools in chat:
model = GenerativeModel(
    "gemini-pro",
    # You can specify tools when creating a model to avoid having to send the
    tools=[weather_tool],
)

# Activate automatic function calling:
afc_responder = AutomaticFunctionCallingResponder(
    # Optional:
    max_automatic_function_calls=5,
```

```
)
chat = model.start_chat(responder=afc_responder)
# Send a message to the model. The model will respond with a function call.
# The SDK will automatically call the requested function and respond to the mo
# The model will use the function call response to answer the original questio
print(chat.send_message("What is the weather like in Boston?"))
```

Evaluation

- To perform bring-your-own-response(BYOR) evaluation, provide the model responses in the **`response`** column in the dataset. If a pairwise metric is used for BYOR evaluation, provide the baseline model responses in the **`baseline_model_response`** column.

```
import pandas as pd
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples

eval_dataset = pd.DataFrame({
        "prompt"  : [...],
        "reference": [...],
        "response" : [...],
        "baseline_model_response": [...],
})
eval_task = EvalTask(
    dataset=eval_dataset,
    metrics=[
            "bleu",
            "rouge_l_sum",
            MetricPromptTemplateExamples.Pointwise.FLUENCY,
            MetricPromptTemplateExamples.Pairwise.SAFETY
    ],
    experiment="my-experiment",
)
eval_result = eval_task.evaluate(experiment_run_name="eval-experiment-run")
```

- To perform evaluation with Gemini model inference, specify the **`model`** parameter with a **`GenerativeModel`** instance. The input column name to the model is **`prompt`** and must be present in the dataset.

```
from vertexai.evaluation import EvalTask
from vertexai.generative_models import GenerativeModel

eval_dataset = pd.DataFrame({
    "reference": [...],
    "prompt"  : [...],
})
result = EvalTask(
    dataset=eval_dataset,
    metrics=["exact_match", "bleu", "rouge_1", "rouge_l_sum"],
    experiment="my-experiment",
).evaluate(
    model=GenerativeModel("gemini-1.5-pro"),
    experiment_run_name="gemini-eval-run"
)
```

- If a **`prompt_template`** is specified, the **`prompt`** column is not required. Prompts can be assembled from the evaluation dataset, and all prompt template variable names must be present in the dataset columns.

```
import pandas as pd
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples
from vertexai.generative_models import GenerativeModel

eval_dataset = pd.DataFrame({
    "context"    : [...],
    "instruction": [...],
})
result = EvalTask(
    dataset=eval_dataset,
    metrics=[MetricPromptTemplateExamples.Pointwise.SUMMARIZATION_QUALITY],
).evaluate(
    model=GenerativeModel("gemini-1.5-pro"),
    prompt_template="{instruction}. Article: {context}. Summary:",
)
```

- To perform evaluation with custom model inference, specify the **`model`** parameter with a custom inference function. The input column name to the custom inference function is **`prompt`** and must be present in the dataset.

```
from openai import OpenAI
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples


client = OpenAI()
def custom_model_fn(input: str) -> str:
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
        {"role": "user", "content": input}
        ]
    )
    return response.choices[0].message.content

eval_dataset = pd.DataFrame({
    "prompt"  : [...],
    "reference": [...],
})
result = EvalTask(
    dataset=eval_dataset,
    metrics=[MetricPromptTemplateExamples.Pointwise.SAFETY],
    experiment="my-experiment",
).evaluate(
    model=custom_model_fn,
    experiment_run_name="gpt-eval-run"
)
```

- To perform pairwise metric evaluation with model inference step, specify the **`baseline_model`** input to a **`PairwiseMetric`** instance and the candidate **`model`** input to the **`EvalTask.evaluate()`** function. The input column name to both models is **`prompt`** and must be present in the dataset.

```
import pandas as pd
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples, Pairw
from vertexai.generative_models import GenerativeModel

baseline_model = GenerativeModel("gemini-1.0-pro")
candidate_model = GenerativeModel("gemini-1.5-pro")

pairwise_groundedness = PairwiseMetric(
    metric_prompt_template=MetricPromptTemplateExamples.get_prompt_template(
        "pairwise_groundedness"
    ),
```

```
    baseline_model=baseline_model,
)
eval_dataset = pd.DataFrame({
    "prompt"  : [...],
})
result = EvalTask(
    dataset=eval_dataset,
    metrics=[pairwise_groundedness],
    experiment="my-pairwise-experiment",
).evaluate(
    model=candidate_model,
    experiment_run_name="gemini-pairwise-eval-run",
)
```

Agent Engine

Before you begin, install the packages with

```
pip3 install --upgrade --user "google-cloud-aiplatform[agent_engines,adk]>=1.
```

First, define a function that looks up the exchange rate:

```
def get_exchange_rate(
    currency_from: str = "USD",
    currency_to: str = "EUR",
    currency_date: str = "latest",
):
    """Retrieves the exchange rate between two currencies on a specified date

    Uses the Frankfurter API (https://api.frankfurter.app/) to obtain
    exchange rate data.

    Returns:
        dict: A dictionary containing the exchange rate information.
            Example: {"amount": 1.0, "base": "USD", "date": "2023-11-24",
                "rates": {"EUR": 0.95534}}
    """
    import requests
    response = requests.get(
        f"https://api.frankfurter.app/{currency_date}",
```

```
        params={"from": currency_from, "to": currency_to},
    )
    return response.json()
```

Next, define an ADK Agent:

```
from google.adk.agents import Agent
from vertexai.agent_engines import AdkApp

app = AdkApp(agent=Agent(
    model="gemini-2.0-flash",        # Required.
    name='currency_exchange_agent',  # Required.
    tools=[get_exchange_rate],       # Optional.
))
```

Test the agent locally using US dollars and Swedish Krona:

```
async for event in app.async_stream_query(
    user_id="user-id",
    message="What is the exchange rate from US dollars to SEK today?",
):
    print(event)
```

To deploy the agent to Agent Engine:

```
vertexai.init(
    project='my-project',
    location='us-central1',
    staging_bucket="gs://my-staging-bucket",
)

remote_app = vertexai.agent_engines.create(
    app,
    requirements=["google-cloud-aiplatform[agent_engines,adk]"],
)
```

You can also run queries against the deployed agent:

```
async for event in remote_app.async_stream_query(
    user_id="user-id",
    message="What is the exchange rate from US dollars to SEK today?",
):
    print(event)
```

Documentation

You can find complete documentation for the Vertex AI SDKs and the Gemini model in the Google Cloud documentation (https://cloud.google.com/vertex-ai/docs/generative-ai/learn/overview)

Contributing

See Contributing (https://github.com/googleapis/python-aiplatform/blob/main/CONTRIBUTING.rst) for more information on contributing to the Vertex AI Python SDK.

License

The contents of this repository are licensed under the Apache License, version 2.0 (http://www.apache.org/licenses/LICENSE-2.0).

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-02-26 UTC.